Hybrid Architectural Paradigms:

Integrating Mamba State Space Models with Recursive Language Models

A Comprehensive Survey and Architectural Proposal

Peter A. Aldrich Jr.
Independent Researcher & Platform Engineer

Preprint — February 2026

Keywords: *State Space Models • Mamba • Selective Scan • Recursive Language Models • Adaptive Computation • PonderNet • Hybrid Architectures • Linear Attention • Weight Sharing • Efficient Inference • Tree-Structured SSMs • Tiny Recursive Models*

Abstract

The Transformer architecture has dominated language modeling since 2017, but its quadratic attention complexity creates fundamental scaling bottlenecks at long sequence lengths, while its single-pass autoregressive generation exposes a fundamental inability to perform non-linear, tree-structured algorithmic reasoning. This paper presents a comprehensive survey and architectural proposal addressing both limitations through the convergence of two distinct architectural lineages: Structured State Space Models (SSMs), most notably the Mamba family, and Recursive Neural Networks reimagined as Recursive Language Models (RLMs) and Tiny Recursive Models (TRMs). We provide exhaustive mathematical foundations for both families, catalog the rapidly expanding landscape of production hybrid SSM-Transformer architectures (Jamba, Nemotron-H, Hymba, Zamba, Hunyuan-TurboS), and propose a concrete hybrid architecture combining a Mamba-2 dominant backbone with sparse shared attention and PonderNet-style adaptive recursive depth looping. We further analyze three distinct hybridization variants: programmatic RLM-Mamba scaffolds, latent recursive operator designs incorporating Mamba-2 within TRM loops, and tree-structured state space models. Empirical evidence demonstrates that models with fewer than 10 million parameters using hybrid Mamba-recursive architectures can outperform 671-billion parameter dense autoregressive models on stringent reasoning benchmarks, while production-scale hybrid deployments achieve 2–3x inference speedup and 10x or greater memory reduction compared to pure Transformers. The proposed design achieves O(n) complexity in sequence length with variable-depth reasoning per token, combining the efficiency of SSMs with the adaptive computation capabilities of recursive models.

1. Introduction: The Architectural Convergence

The rapid and unprecedented evolution of deep learning for natural language processing, computer vision, and complex algorithmic reasoning has been predominantly driven by the Transformer architecture and its core self-attention mechanism. Since its introduction, the Transformer has established a robust, highly parallelizable paradigm for foundation models, enabling the scaling of Large Language Models (LLMs) to hundreds of billions of parameters. However, the foundational mechanisms that propelled the Transformer to dominance inherently contain the seeds of its most significant computational and cognitive limitations.

As sequence lengths expand from thousands to millions of tokens to accommodate entire codebases, exhaustive legal documents, and extensive scientific corpora, the quadratic memory and computational complexity of the attention matrix becomes a severe, often intractable bottleneck. Furthermore, as these models are increasingly tasked with deliberate planning, formal logic, and mathematical proofs, their reliance on single-pass autoregressive generation exposes a fundamental inability to perform non-linear, tree-structured algorithmic reasoning.

To resolve these orthogonal but deeply intersecting challenges, the artificial intelligence research landscape is witnessing the resurgence, modernization, and eventual hybridization of two distinct architectural lineages. Selective State Space Models (SSMs), most notably epitomized by the Mamba architecture, offer linear-time sequence modeling with selective state propagation, effectively solving the context length and throughput bottlenecks of traditional Transformers by discarding the need for an

exhaustive Key-Value (KV) cache. Concurrently, Recursive Language Models (RLMs) and Tiny Recursive Models (TRMs) provide the structural computational scaffold necessary for hierarchical data processing, dynamic context manipulation via isolated execution environments, and latent iterative refinement through deep recursive backpropagation.

This comprehensive paper provides a mathematically rigorous analysis of the theoretical foundations, architectural formulations, and empirical implementations of hybrid model architectures that integrate Mamba LLMs with recursive computational paradigms. By fusing the linear-time sequential efficiency of Selective SSMs with the compositional, tree-structured reasoning of recursive models, researchers are synthesizing an entirely new class of foundation models. These hybrid architectures promise to preserve reasoning capabilities within a recursive scaffold while leveraging state space models to enhance candidate coverage, maintain high-quality predictions, and process theoretically infinite context windows without suffering from cognitive degradation.

2. The Bottlenecks of Autoregressive Attention and the Context Rot Phenomenon

2.1 Quadratic Complexity and KV Cache Constraints

The vanilla Transformer is fundamentally an encoder-decoder or decoder-only architecture built upon the Scaled Dot-Product Attention mechanism. This function operates on a set of input vectors and produces a new set of vectors where each output is a weighted sum of the values, with the weights determined by the similarity between queries and keys. While this all-to-all communication allows the model to capture rich context across a sequence, it scales quadratically with respect to the sequence length. Processing a sequence of length L requires $O(L^2)$ time and memory, making the ingestion of ultra-long documents prohibitively expensive.

To mitigate this during inference, Transformers cache the keys and values (the KV cache) of previously generated tokens. However, as the context window grows, the memory footprint of the KV cache frequently exceeds the capacity of High Bandwidth Memory (HBM) on modern GPUs, severely degrading inference throughput and increasing latency. Concretely, Llama-2-7B's KV cache consumes approximately 0.5 MB per token. At 128K tokens, this amounts to 64 GB, exceeding the capacity of most consumer and many datacenter GPUs. By contrast, Jamba's hybrid design requires only 4 GB for 256K tokens, a 16x reduction that fits comfortably on a single 80 GB GPU. Beyond approximately 73K tokens on consumer GPUs, SSMs are the only viable architecture without memory offloading techniques.

2.2 Context Rot and Attention Degradation

Beyond pure hardware constraints, the push toward longer contexts has revealed a profound cognitive limitation known as "Context Rot" or attention degradation. Between 2023 and 2025, the industry witnessed an arms race to expand context windows, pushing limits from 4,096 tokens to theoretical windows exceeding 10 million tokens. This expansion fueled the assumption that a model could simply be fed an entire library and instantly master its content. The operational reality, however, demonstrated that as context length increases, the model's capacity to reason, synthesize, and establish complex connections between distant facts collapses.

The quadratic attention mechanism, or even its linear approximations, struggles to discriminate signal from noise when the signal-to-noise ratio becomes extremely low. Paradoxically, the model becomes less capable of reasoning the more information it is fed. Simplistic "Needle-in-a-Haystack" benchmarks often mask this limitation by only testing raw retrieval rather than the synthesis of multiple, deeply buried concepts. Furthermore, because Transformers lack a native, dynamic mechanism to actively "forget" irrelevant information, the attention distribution becomes increasingly uniform and diluted over millions of tokens.

2.3 The Autoregressive Reasoning Limitation

Simultaneously, the autoregressive nature of the Transformer decoder is fundamentally ill-suited for logic puzzles, complex planning, or formal proofs. Autoregressive models make one irreversible decision at a time, generating tokens sequentially from left to right. In rigorous algorithmic tasks, a single early error invalidates the entire downstream solution. Transformers cannot naturally backtrack, explore branching decision trees, or iteratively refine a hidden state before committing to an output token. Their fixed depth places them in computational complexity classes such as AC^0 or TC^0, which prevents them from natively solving problems that require polynomial time execution, effectively meaning they are not Turing-complete in a purely end-to-end, single-pass formulation.

These dual limitations—Context Rot over long sequences and the inability to perform non-linear algorithmic refinement—form the primary motivation for hybridizing Mamba and recursive architectures.

3. Foundations of the Mamba Architecture: Selective State Spaces

The Mamba architecture, introduced to address the aforementioned computational inefficiencies, represents a paradigm shift in sequence modeling. It transitions the field away from explicit attention-based token mixing toward implicit, input-dependent state space dynamics. Mamba builds upon a long history of continuous-time models, recurrent neural networks, and structured parameter initialization to achieve what was previously thought impossible: the parallelizable training of a convolutional network with the constant-time inference of a recurrent network.

3.1 The Continuous and Discrete State Space Formulation

The mathematical foundation of Mamba is derived from the continuous-time State Space Model (SSM). This mathematical framework maps a 1-dimensional continuous input signal x(t) to a 1-dimensional output signal y(t) through an intermediate, implicit hidden state h(t). The continuous dynamical system is governed by a set of linear ordinary differential equations:

$$x'(t) = A \cdot x(t) + B \cdot u(t)$$

$$y(t) = C \cdot x(t) + D \cdot u(t)$$

In this formulation, $A \in \mathbb{R}^{N \times N}$ represents the state transition matrix, $B \in \mathbb{R}^{N \times 1}$ is the input projection matrix, $C \in \mathbb{R}^{1 \times N}$ is the output projection matrix, and D functions as a direct skip connection from the input to the output.

Because deep learning models operate on discrete sequences of tokens rather than continuous physical signals, these continuous parameters must be discretized. Using a step size parameter Δ, which represents the resolution or sampling rate of the input, the continuous matrices A and B are transformed into their discrete counterparts using the zero-order hold (ZOH) rule, which assumes the input is constant during the sampling interval:

$$\bar{A} = exp(\Delta \cdot A)$$

$$\bar{B} = (\Delta A)^{-1}(exp(\Delta A) - I) \cdot \Delta B$$

This yields the discrete recurrence equations that define the forward pass of a basic structured SSM:

$$h_t = \bar{A} \cdot h_{t-1} + \bar{B} \cdot x_t$$

$$y_t = C \cdot h_t$$

S4 (Gu, Goel, and Ré; ICLR 2022) made this practical through two key innovations. First, HiPPO initialization constrains the state matrix A to project continuous input history onto Legendre polynomials, providing a mathematically principled basis for long-range memory. Second, the Normal Plus Low-Rank (NPLR) parameterization of A enables efficient convolution-mode training via FFT in O(L log L) time. S4 was the first architecture to solve the Path-X benchmark at sequence length 16,384 and achieved state-of-the-art across all six Long Range Arena tasks.

In earlier iterations of structured SSMs such as S4, the matrices were strictly Linear Time-Invariant (LTI), meaning they remained constant and fixed across all time steps. The LTI property was highly desirable for training because it allowed the sequential recurrence to be mathematically unrolled into a massive, 1-dimensional global convolution kernel using the Fast Fourier Transform. However, this LTI property was also their greatest weakness when applied to natural language.

3.2 The Selective Mechanism (S6) and Content-Based Reasoning

The critical innovation that elevated Mamba to rival Transformers was the rejection of the LTI constraint in favor of a time-varying, input-dependent mechanism known as Selective State Spaces (S6). Traditional LTI SSMs struggled severely with discrete, information-dense modalities like text because they could not dynamically alter their dynamics based on the input. They were forced to process irrelevant filler words, punctuation, and critical context with the exact same mathematical weight and state transition logic.

Mamba (Gu and Dao, December 2023) broke this constraint by making key parameters input-dependent, creating the S6 mechanism. In this formulation, B_t, C_t, and the step size Δ_t are all computed as linear projections of the current input x_t:

$$B_t = Linear^b(x_t),\ C_t = Linear^c(x_t),\ \Delta_t = softplus(Linear\Delta(x_t))$$

The step size Δ acts as a selective gate: large values emphasize the current input (effectively resetting state), while small values preserve existing memory—functionally analogous to the forget gate in LSTMs. By parameterizing these values based on the exact token being processed, Mamba achieves true content-based reasoning. This allows the model to selectively propagate important information across the sequence length, or actively reset its state to forget irrelevant history.

This mechanistic filtering directly addresses the Context Rot phenomenon. By actively discarding noise, Mamba maintains high signal integrity over sequences stretching up to 1 million tokens, establishing a near-unbounded context capacity. Furthermore, this selection mechanism provides a mathematically principled generalization of the heuristic gating mechanisms found in legacy RNNs like LSTMs and GRUs, elevating gated recurrence into a stable, continuous state space framework.

3.3 Hardware-Aware Parallel Scan Optimization

Because input-dependent parameters break the convolution-mode training path, Mamba introduced a hardware-aware parallel scan algorithm. The key insight is that the state recurrence defines an associative binary operator on ($\bar{A}$, $\bar{B}\cdot x$) tuples, enabling a work-efficient parallel prefix scan with O(L) work and O(log L) span.

Modern GPUs feature a specific memory hierarchy consisting of a massive but relatively slow High Bandwidth Memory (HBM) and a much smaller but exceptionally fast SRAM. Mamba minimizes costly I/O operations through kernel fusion. Instead of materializing the expanded hidden states h□ in the slow HBM, the algorithm loads the input and the SSM parameters directly into the fast SRAM. It performs the continuous-to-discrete transformation and the recurrent parallel scan state updates entirely within the SRAM, writing only the final output y□ back to the HBM. This yields an O(N) reduction in memory I/O versus naive implementation, directly inspired by FlashAttention's IO-aware design philosophy. This architectural optimization allows Mamba to train significantly faster than previous SSMs and achieve up to a 5x higher inference throughput than equivalently sized autoregressive Transformers.

3.4 Mamba-2 and State Space Duality

Mamba-2 (Dao and Gu, ICML 2024) revealed a deep mathematical connection between SSMs and attention through the Structured State Space Duality (SSD) framework. By constraining the diagonal A matrix to a scalar times identity (a single scalar a□ shared across all N state dimensions per timestep), the SSM computation becomes exactly equivalent to a structured masked matrix multiplication:

$$M = L \odot (C \cdot B^T)$$

$$Y = M \cdot X$$

where L is a lower-triangular matrix defined by cumulative products of the a□ values. When all a□ equal 1, L becomes the standard causal mask and SSD reduces to causal linear attention, with the correspondence (C, B, X) mapping to (Q, K, V).

This duality enables a block decomposition algorithm combining both computation modes: quadratic (attention-like) within chunks of size Q using matrix multiplications on tensor cores, and linear (recurrent) between chunks using state propagation. On A100 GPUs, this exploits 312 TFLOPS BF16 matmul capacity versus just 19 TFLOPS for scalar operations—a potential 16x speedup. Mamba-2 achieves 2–8x faster training than Mamba-1, supports state dimensions of N = 64–256 (versus N = 16), introduces multi-head SSMs analogous to multi-head attention, and enables tensor parallelism with just one all-reduce per layer.

4. Paradigms of Recursive Computation in Deep Learning

While Mamba elegantly solves the 1-dimensional sequence modeling problem, human reasoning, natural language syntax, and algorithmic logic are inherently compositional, hierarchical, and multi-dimensional. To process this structural complexity, the deep learning community has turned toward Recursive Neural Networks (RvNNs) and their modern instantiations. Recursive processing encompasses three distinct families, each offering complementary capabilities when integrated with SSM architectures.

4.1 Recursive Neural Networks and Topological Data Structures

Traditional Recursive Neural Networks are a distinct class of deep neural networks created by applying a shared set of weights recursively over a structured input. Unlike standard RNNs, which operate on a linear chain of time steps combining the current input x_t and the previous hidden state h_{t-1}, RvNNs are designed to traverse a given topology—such as a syntactic parse tree or a molecular graph—in strict topological order.

RvNNs process hierarchical inputs by combining child node representations into parent node representations in a bottom-up fashion. For a binary tree structure, the representation of a parent node h_p is computed from its left child h_l and its right child h_r:

$$h_p = f(W_l \cdot h_l + W_r \cdot h_r + b)$$

Tree-LSTMs (Tai, Socher, and Manning; ACL 2015) extended this with gating mechanisms, using per-child forget gates that selectively propagate information through tree structure. While these models require pre-parsed inputs and resist GPU batching, their principle—hierarchical composition of representations—informs how SSM states could be organized at multiple structural levels.

4.2 Weight-Shared Depth Looping

The most directly applicable recursive paradigm applies the same transformation function repeatedly. ALBERT (Lan et al., ICLR 2020) demonstrated that sharing all Transformer parameters across 12 layers yields an 18x parameter reduction (18M vs. 334M for BERT-large) while improving downstream performance—establishing that weight-tied depth is viable at scale. Universal Transformers (Dehghani et al., ICLR 2019) combined weight sharing with Adaptive Computation Time (ACT), where a sigmoid halting unit per position controls iteration count, yielding Turing completeness and superior algorithmic generalization.

Recent work has dramatically advanced this paradigm. Huginn (Geiping et al., NeurIPS 2025) implements a three-part architecture—Prelude (embedding), Recurrent Block (shared-weight Transformer iterated T times), and Coda (output)—achieving emergent reasoning capabilities at 3.5B parameters trained on 800B tokens. With increased test-time recurrence (effectively 50B parameter compute), Huginn shows dramatic improvements on math and coding benchmarks. Ouro/LoopLM (Zhu et al., ByteDance 2025) scales this approach to 7.7T training tokens, demonstrating that a 1.4B looped model matches 3–4B standard Transformers and a 2.6B model matches 8–12B models.

Deep Equilibrium Models (DEQ) by Bai, Kolter, and Koltun (NeurIPS 2019) take this concept to the limit: instead of fixed iterations, they solve for the fixed point $z^* = f(z^*, x)$ using root-finding algorithms such as Broyden's method, equivalent to infinite-depth weight-tied networks with O(1) memory via implicit differentiation, achieving up to 88% memory reduction versus explicit deep models.

4.3 The Infinite Context Paradigm: Recursive Language Models (RLMs)

To transcend the limitations of both fixed-topology RvNNs and the Context Rot of linear Transformers, researchers at MIT introduced Recursive Language Models (RLMs) in late 2025. RLMs represent a general-purpose inference paradigm designed to dramatically scale effective input and output lengths by treating context not as internal data, but as a programmable environment.

In the RLM paradigm, an arbitrarily long user prompt or document is not fed directly into the neural network's context window. Instead, the context is stored as a variable within an external Python REPL (Read-Eval-Print Loop) environment, similar to a background Jupyter Notebook. The architecture isolates a "root LM" (operating at recursion depth 0) from the full text. When the root LM receives a query, it writes programmatic code (using Python string manipulation or regular expressions) to peek into, decompose, and transform the context variable.

Crucially, the root LM possesses the operational capability to invoke exact sub-instances of itself—recursive calls at depth 1, depth 2, and beyond—to process specific chunks of the context programmatically. Each recursive call spawns a fully isolated sub-RLM instance with its own clean REPL environment. The sub-RLM processes the narrowed subset of data based on a refined sub-query and returns the synthesized output back to the caller's environment. This tree-like recursive decomposition of tasks allows the model to process theoretically infinite context windows, as no single neural forward pass is burdened with the entire sequence.

4.4 Adaptive Computation and Pondering

PonderNet (Banino et al., DeepMind 2021) reformulated adaptive computation with a proper probabilistic framework. The halting probability at each step n defines a geometric distribution:

$$p_n = \lambda_n \cdot \prod(1 - \lambda_j), \text{ for } j < n$$

Training uses expected loss weighted by p_n plus a KL divergence regularizer against a geometric prior. PonderNet provides unbiased, low-variance gradients (versus ACT's biased estimates) and achieves perfect accuracy on extrapolation tasks where ACT fails.

COCONUT (Hao et al., Meta 2024) demonstrates that iterative reasoning can occur in continuous latent space rather than discrete token space. The LLM's last hidden state is fed back as the next input embedding, enabling breadth-first search over multiple reasoning paths simultaneously—outperforming chain-of-thought on tasks requiring backtracking.

4.5 Latent Iterative Refinement: Tiny Recursive Models (TRMs)

Parallel to the programmatic recursion of RLMs, structural recursion has been applied to internal latent state refinement. The development of the Hierarchical Reasoning Model (HRM) and the subsequent Tiny Recursive Model (TRM) demonstrates that deep, recursive computation on internal latent states can drastically outperform standard autoregressive parameter scaling.

TRMs utilize a remarkably shallow neural network backbone—often consisting of just two layers comprising a mere 7 million parameters—but apply this network recursively to iteratively refine a solution space. Unlike autoregressive models that commit to an output token immediately, TRMs separate

the internal "reasoning" latent state from the final "answer" latent state. The network loops over the reasoning state multiple times, updating its hypotheses and refining its internal logic. During training, full backpropagation is applied through the final unrolled recursion steps, forcing the network to learn genuine algorithmic computation rather than merely memorizing training data.

The empirical results of this paradigm are staggering. A 7-million parameter TRM achieved a 45% test accuracy on the highly complex ARC-AGI-1 reasoning benchmark, drastically outperforming densely scaled 671-billion parameter models (such as DeepSeek R1 or Gemini 2.5 Pro) that score near 0% on the same tasks. This underscores a vital theoretical insight: the algorithmic structure of recursive computation provides a far superior inductive bias for logic, deduction, and planning tasks than the sheer scale of linear sequence models.

4.6 The Synthesis Opportunity

All recursive approaches share a common mathematical core: iterative application of a parameterized function $h(t) = f\theta(h(t-1), x)$. SSMs naturally express this through their recurrent state dynamics. The key insight is that SSMs already implement a form of recursive processing along the sequence dimension; adding recursive depth (iterating the same SSM block multiple times) creates a two-dimensional computation grid—sequence-wise recurrence for efficient context processing, depth-wise recurrence for adaptive reasoning.

5. Theoretical Synergy: The Dynamical Systems Framework

The orthogonal strengths of Mamba (unmatched hardware efficiency, linear sequence processing, and dynamic state selection) and Recursive Models (hierarchical reasoning, infinite context decomposition, and algorithmic latent refinement) present a compelling case for architectural hybridization. The theoretical justification for merging these paradigms is rigorously formalized within the Dynamical Systems Framework (DSF).

Recent theoretical evaluations of linear RNNs, which encompass both Mamba and gated linear attention models, demonstrate that while they excel at sequence tracking and continuous state updates, their fixed architectural depth severely limits their computational complexity class. Pure linear RNNs are not natively Turing-complete. They fundamentally struggle with dynamic, non-linear algorithmic execution that requires polynomial time processing, such as complex parity evaluations or modulus counting. To solve these algorithmic tasks, the state transition matrices must be augmented, either by extending eigenvalue ranges into negative domains to allow for sharp, oscillating state transitions, or by embedding the SSM within a structural recursive loop.

By embedding a Mamba block within a recursive scaffold—or conversely, defining a recursive tree structure where the information flow along the edges is governed by Selective SSM dynamics—architectures can achieve unparalleled synthesis. This hybrid approach captures the local, sequential coherence and hardware efficiency of state space models while simultaneously exploiting the global, compositional reasoning and tree-structured logic of recursive networks.

Analysis from Albert Gu and colleagues confirms that FLOP-for-FLOP, SSMs model language better than Transformers in terms of perplexity, particularly on byte-level (untokenized) data where global

attention requires 2x the FLOPs for comparable perplexity. This suggests that replacing most Transformer layers with SSM layers improves both efficiency and raw modeling quality. However, downstream algorithmic capabilities—in-context learning, retrieval, format adherence—still favor attention, justifying the hybrid approach.

6. The Hybrid Architecture Landscape in 2025

The field has converged on hybrid Mamba-Transformer as the dominant next-generation architecture, with every major AI lab deploying production models. Three distinct hybridization patterns have emerged from this rapid development.

6.1 Serial Block Interleaving

The most common pattern alternates SSM and attention layers at fixed ratios. Jamba (AI21 Labs, ICLR 2025) established the template: a 1:7 attention-to-Mamba ratio with Mixture of Experts applied every 2 layers, scaling to 398B total (94B active) parameters with 256K token context and only 4 GB KV cache (versus 32 GB for Mixtral). A critical finding was that even a single attention layer out of eight restores in-context learning capabilities that pure Mamba lacks, while pure Mamba layers handle efficient sequential processing.

NVIDIA's Nemotron-H family validated this at massive scale: Nemotron-H-56B uses 54 Mamba-2 layers plus 10 self-attention layers (approximately 8% attention), matching Qwen-2.5-72B with up to 3x faster inference. Their controlled 8B comparison (identical data, same compute) showed the hybrid exceeded the pure Transformer on all 12 standard benchmarks by an average of +2.65 points. On 23 long-context tasks, the hybrid matched or exceeded the Transformer. Pure SSMs matched Transformers on commonsense reasoning and language modeling but consistently failed on phonebook lookup (associative recall) and lagged on MMLU 5-shot in-context learning—gaps that even minimal attention layers close.

Hunyuan-TurboS (Tencent, 2025) deployed this pattern at 560B total parameters with 57 Mamba-2 plus 7 attention layers, achieving top-7 on the LMSYS Chatbot Arena with 1.8x speedup over their pure Transformer baseline. Multiple independent research groups converge on the same finding: the optimal ratio falls between 3:1 and 10:1 SSM-to-attention layers. Layer placement also matters—attention should be evenly distributed throughout the model, and SSM layers should process inputs first, as placing attention at the front degrades performance.

6.2 Parallel Within-Layer Fusion

Hymba (NVIDIA, ICLR 2025) fuses attention heads and Mamba SSM heads within the same layer in parallel, using a 5:1 SSM-to-attention parameter ratio. At 1.5B parameters, Hymba surpasses Llama-3.2-3B with 11.67x smaller cache and 3.49x higher throughput. The parallel design allows attention and SSM heads to serve complementary, adaptive roles that vary by layer and input.

6.3 Shared-Weight Attention

Zamba (Zyphra, 2024) takes parameter efficiency further: a Mamba backbone with a single shared attention block (identical weights reused every 6 Mamba layers). Zamba2 extends this to two shared blocks in an alternating pattern with LoRA projection matrices for per-position specialization. The core insight—that the vast majority of attention parameters are redundant—means a single set of attention weights captures the global information Mamba misses. Zamba2-1.2B outperforms Gemma2-2B despite being half the size.

Table 1: Production Hybrid SSM-Transformer Architectures (2024–2025)

Model	Parameters	SSM:Attn	Context	Key Result
Jamba	398B / 94B active	7:1	256K	4 GB KV cache (vs 32 GB Mixtral)
Nemotron-H	56B	~5:1	128K	+2.65 avg pts over Transformer
Hymba	1.5B	5:1*	128K	11.67x cache reduction, 3.49x throughput
Zamba2	1.2–7.4B	6:1†	4K	Outperforms Gemma2-2B at half size
Hunyuan-TS	560B total	~8:1	256K	1.8x speedup, top-7 LMSYS Arena
Bamba	9B	~3:1	4K	Matches Llama-3.1-8B on 7x fewer tokens

** Parallel fusion within layer. † Shared attention weights.*

7. Designing the Hybrid Architecture: Three Variants

7.1 Variant I: The Programmatic RLM-Mamba Scaffold

The first variant integrates a pure Mamba-2 model, or a Mamba-Transformer hybrid (such as Jamba), as the core cognitive engine for the Recursive Language Model (RLM) framework. In the standard RLM paradigm, the root LM must spawn numerous independent sub-LMs in isolated REPL environments. When using traditional Transformers for this task, the memory overhead of managing individual KV caches for dozens of concurrent recursive calls becomes a severe memory bottleneck, limiting the depth of the recursion tree.

Integrating Mamba as the underlying engine completely eliminates this KV cache memory footprint. Because Mamba models the sequence via a compressed hidden state h_t of fixed size, it requires only $O(1)$ memory per sequence regardless of how many tokens it processes. As the RLM dynamically chunks the unbounded context variable using Python regex and recursively feeds it to depth-1 and depth-2 sub-instances, the Mamba engine processes these chunks sequentially with maximal efficiency.

This programmatic hybrid explicitly utilizes the Selective SSM mechanism to rapidly scan through massive code blocks or text chunks generated within the REPL environment. The input-dependent Δ_t allows the sub-RLM to selectively remember critical variables while entirely filtering out standard Python boilerplate or repetitive formatting strings. Once the sub-RLM extracts the necessary insight, it compiles the finding and returns the result to the parent node, allowing the recursion tree to collapse back to the root LM with perfectly preserved signal fidelity.

7.2 Variant II: Latent Recursive Operator Design

The most rigorous empirical exploration of Mamba-Recursive hybridization occurs at the operator level, specifically within the Tiny Recursive Model (TRM) architecture. TRMs traditionally rely on attention-heavy Transformer blocks as the per-step operator to iteratively refine latent states within the reasoning loop. However, recent studies have demonstrated that Mamba-2 can effectively replace or augment standard Transformer blocks within the recursive loop without degrading cognitive capability.

Because Mamba-2 excels at sequential dependencies but processes information causally in one direction, researchers designed specific hybrid operators to capture bidirectional dependencies essential for grid-based algorithmic reasoning tasks. Two primary operator pipelines have been established. The **TR-mamba2attn** variant consists of two consecutive Mamba-2 blocks, followed by a standard Self-Attention block and a Multi-Layer Perceptron (MLP) block. During each iteration of the recursive loop, the Mamba-2 blocks efficiently process the flattened sequence while the subsequent Attention block facilitates explicit all-to-all cross-position communication crucial for resolving bidirectional logic dependencies.

The **TR-mamba2mlpt** variant replaces the quadratic Attention block with an MLP-t (an MLP operating on the transposed sequence dimension), providing all-to-all cross-position communication without the heavy quadratic memory overhead of self-attention. It relies heavily on the preceding Mamba-2 blocks to have already aligned the sequential logic and filtered the noise.

Empirical validation reveals that substituting pure attention with a Mamba-2 hybrid operator within a recursive scaffold significantly improves candidate coverage. The inherent sequential processing of Mamba-2 contributes highly distinct solution trajectories compared to global attention, increasing the diversity of generated candidates without degrading the model's ability to select the best answer. Researchers observed that the Mamba operator and the Attention operator solve partially disjoint sets of logic puzzles, confirming that their complementary mixing within a recursive scaffold maximizes the total reasoning capability. Post-normalization layers applied immediately after the Mamba blocks are absolutely critical for stabilizing the recursive computation, as the compounding nature of the continuous SSM state updates across multiple iterations causes the latent state to diverge without normalization.

7.3 Variant III: Tree-Structured State Space Models

Moving beyond sequential latent refinement, architectures such as MambaTree and Tree-Structured Language Modeling (TSLM) map the Mamba state space directly onto complex, non-linear graph topologies. Standard Mamba models are constrained by the inherent geometric constraints of 1-dimensional sequences, but underlying syntactic structures and spatial relationships are fundamentally hierarchical.

To formulate a Tree-Structured Mamba architecture, the standard discrete SSM equations are adapted to a tree topology. Let T = (V, E) represent a directed tree where V are nodes and E are edges. For any node v, let p(v) denote its unique parent. The hidden state of node v, denoted h^v, is computed recursively from its parent's hidden state:

$$h^v = \bar{A}(x^v) \cdot h^{p(v)} + \bar{B}(x^v) \cdot x^v$$

The selection mechanism remains tied to the local input at each node, allowing the model to dynamically alter the transition matrices as it traverses down a branch. For bottom-up compositionality where a node v has multiple children C(v), the Mamba selection mechanism performs a weighted, content-aware aggregation of child states before applying the state transition.

In the TSLM paradigm, the model learns systematic exploration by training on complete search trees, encompassing both successful logic paths and failed attempts. During inference, the selective SSM state is duplicated at branch points, allowing independent parallel scans down different paths of the decision tree without redundant recomputation of the shared prefix.

8. Proposed Hybrid Mamba-Recursive Architecture

Drawing on the surveyed literature, we propose a concrete architecture combining Mamba-2's efficient sequence processing with recursive depth looping and adaptive computation. The design synthesizes three independently validated insights into a unified model.

8.1 High-Level Design

The architecture consists of three components inspired by the Huginn pattern. The **Prelude (P)** contains 4 standard Mamba-2 blocks that embed input tokens into the latent representation space; these are not weight-shared and serve as feature extractors. The **Recursive Core (R)** is a block of K Mamba-2 layers (K = 4–8) with 1 shared attention layer, iterated T times with shared weights; input embeddings are re-injected at each iteration, and a PonderNet-style halting mechanism controls T per token. The **Coda (C)** contains 2 standard Mamba-2 blocks plus 1 attention layer that process the final latent state and produce next-token logits.

Total unique parameters scale as P_prelude + P_core + P_coda, where the recursive core's parameters are counted only once regardless of iteration count T—yielding ALBERT-style parameter efficiency with Huginn-style adaptive reasoning depth.

8.2 The Recursive Core

Each iteration t of the recursive core processes the state through RMSNorm, then applies the K shared-weight layers (K−1 Mamba-2 blocks plus 1 shared attention block), with MLP sub-layers after each. The halting mechanism follows PonderNet:

$$\lambda_t = \sigma(W_halt \cdot pool(s) + b_halt)$$

$$p_t = \lambda_t \cdot (1 - cumulative_halt)$$

The geometric distribution p_t defines the halting probability. Training uses expected loss weighted by p_t with a geometric prior regularizer. During inference, early exit occurs when cumulative halting probability exceeds 0.99.

8.3 Attention Integration Strategy

Following Zamba's shared-weight approach, the recursive core contains a single attention block with shared parameters across all iterations. This is augmented with per-iteration LoRA adapters (rank 16–32)

for specialization. The attention receives a concatenation of the current state and original embeddings (Zamba's key insight), helping maintain input fidelity across recursive iterations. This provides global information access while keeping parameter overhead minimal.

8.4 Architecture Pseudocode

```
class HybridMambaRecursiveModel(nn.Module):
  def forward(self, input_ids, targets=None):
    e0 = self.embed(input_ids)
    h = e0
    # Prelude: 4 unique Mamba-2 blocks
    for block in self.prelude:
      h = h + block(RMSNorm(h))
    # Recursive Core with PonderNet
    cum_halt = zeros_like(h[:,:,:1])
    for t in range(self.max_T):
      h_iter = RMSNorm(h)
      for k, layer in enumerate(self.core):
        if k == attn_pos:
          h_iter += SharedAttn(h_iter, e0, t)
        else:
          h_iter += Mamba2Block(h_iter)
        h_iter += MLP(RMSNorm(h_iter))
      h = h + h_iter
      lam = sigmoid(W_halt @ pool(h))
      p_t = lam * (1 - cum_halt)
      cum_halt += p_t
      if not training and cum_halt.min() > 0.99:
        break
    # Coda: 2 Mamba-2 + 1 Attention + LM head
    for block in self.coda:
      h = h + block(RMSNorm(h))
    return self.lm_head(h)
```

8.5 Complexity Analysis

For a model with D = d_model, N = state dimension, L = sequence length, K = layers per iteration, T = average iterations, and A = attention layers per iteration:

Table 2: Computational Complexity Comparison

Component	Train FLOPs	Inference Memory
Mamba-2 (per iter)	$O(K \cdot L \cdot D \cdot N)$	$O(D \cdot N)$ fixed
Shared Attn (per iter)	$O(A \cdot L \cdot W \cdot D)$	$O(A \cdot W \cdot D)$
Full Hybrid (T iters)	$O(T \cdot K \cdot L \cdot D \cdot N)$	$O(D \cdot N + A \cdot W \cdot D)$
Equiv. Transformer	$O(T \cdot K \cdot L^2 \cdot D)$	$O(T \cdot K \cdot L \cdot D)$

At L = 128K, D = 4096, N = 128, K = 6, T = 4, and W = 4096, the hybrid requires approximately 1.6T FLOPs per forward pass versus approximately 1,600T FLOPs for an equivalent-depth Transformer with full attention—a roughly 1000x reduction at long sequence lengths.

9. Hardware Optimization, Speculative Decoding, and Training Dynamics

9.1 Speculative Decoding with SSMs

One of the most significant operational advantages of SSMs is their native compatibility with Speculative Decoding. Because the state in an SSM implicitly summarizes all past data without needing to manually cache or re-process tokens, tree-based speculative decoding becomes exponentially more efficient than in Transformers. In a Tree-Structured hybrid model, specific algorithms exploit the structure of accumulated state transition matrices to verify entire branches of a token tree simultaneously. The hardware-aware implementation involves duplicating the SRAM state at decision junctions and running parallel prefix sums along the distinct drafting branches, yielding massive inference speedups with minimal memory overhead.

9.2 Numerical Stability and Training Protocols

Training recursive SSM architectures requires strict adherence to precise numerical stability protocols. SSM parameters must be stored in fp32 even when using mixed-precision training—SSMs are sensitive to their recurrent dynamics, and fp16 can cause state drift or collapse. RMSNorm before every SSM and attention layer is essential for large-scale stability. For recursive depth specifically, residual connections must be carefully scaled to prevent gradient explosion across iterations; Huginn addresses this by backpropagating through only the last k iterations.

Mamba-2's GroupNorm (not LayerNorm) enables tensor parallelism, but group size must exceed 256 for accurate statistics. A notable counter-intuitive finding from the Jamba experiments: Mamba-1

combined with attention outperforms Mamba-2 combined with attention in hybrid settings, despite Mamba-2's standalone superiority. The extra structural constraints of SSD may interact poorly with attention layers in certain configurations.

9.3 Progressive Training Strategy

Evidence from multiple production deployments suggests a multi-phase approach. Phase 1 trains the full architecture with T fixed at a low value (e.g., T = 2) on standard language modeling data for the majority of compute budget. Phase 2 enables adaptive halting and increases the maximum T, training on a curriculum that progressively introduces harder examples. Phase 3 anneals on high-quality data with rapid learning rate decay—Zamba's recipe uses exponential decay from 1.1e-4 to 1e-7, which outperformed linear decay schedules.

The MOHAWK framework (Bick et al., NeurIPS 2024) offers an alternative: distill from an existing Transformer into the SSM-attention hybrid using three phases (matrix orientation, hidden-state alignment, end-to-end distillation), requiring only 3B tokens versus trillions for from-scratch training. Production deployments consistently use AdamW with cosine learning rate scheduling, and NVIDIA achieved 29.9% model FLOPs utilization for their 8B hybrid on 1024 H100 GPUs, nearly matching the Transformer's 30.7%.

10. Empirical Evaluations and Cross-Domain Applications

10.1 Algorithmic Reasoning and Logical Compositionality

The ARC-AGI benchmark serves as the primary arena for evaluating latent reasoning and spatial logic. TRM architectures augmented with Mamba-2 operators have established new state-of-the-art metrics for small-parameter models. NVIDIA's controlled 8B-scale experiment (identical data, 3.5T tokens) provides additional strong evidence: their Mamba-2-Hybrid exceeded the pure Transformer on all 12 standard short-context benchmarks, with MMLU 3.5 points higher. On 23 long-context tasks, the hybrid matched or exceeded the Transformer.

Table 3: Benchmark Performance on Complex Reasoning Tasks

Model	Params	ARC-AGI-1	Sudoku-Ext.	Maze-Hard
DeepSeek R1 / Gemini 2.5	> 300B	~4.9%	< 10.0%	< 15.0%
HRM	27M	40.3%	> 55.0%	75.0%
Pure TRM-Attention	7M	44.6%	85.1%	85.3%
TR-Mamba2Attn Hybrid	~7M	46.6%	87.4%	87.0%

The TR-Mamba2Attn hybrid with ~7M parameters outperforms 300B+ parameter autoregressive models, driven by the +2.0% improvement on pass@2 metrics intrinsic to SSM integration.

The TR-Mamba2attn hybrid explicitly exhibits the "candidate coverage" phenomenon. While its top-1 selection remains comparable to pure attention-based TRMs, the Mamba-driven sequential processing generates a significantly more diverse set of correct solution trajectories. By stratifying the difficulty of evaluation metrics, researchers observed that the Mamba operator and the Attention operator solve partially disjoint sets of logic puzzles, confirming their complementary nature.

10.2 Hierarchical Vision and Spatial Processing

The hybrid Mamba-Recursive paradigm has profoundly impacted computer vision. MambaTree dynamically generates a tree topology based on spatial relationships and input features, significantly outperforming existing linear SSMs on ImageNet-1K and complex detection tasks. Mamba-CNN embeds a lightweight, Mamba-inspired SSM gating mechanism into a hierarchical recursive convolutional backbone for facial beauty prediction, achieving a state-of-the-art Pearson Correlation of 0.9187 on the SCUT-FBP5500 benchmark. HG-Mamba employs bidirectional Spectral and Spatial Mamba modules for hyperspectral imaging, demonstrating the versatility of integrating recursive spatial structures with state space models.

10.3 Throughput, Memory, and Scaling Advantages

The efficiency advantage of hybrid architectures compounds with sequence length. At short sequences (fewer than 2K tokens), optimized Transformers remain 1.8x faster due to mature CUDA optimization. The crossover occurs around 2–4K tokens, after which SSM-dominant hybrids increasingly dominate. At 57K tokens on RTX 4090, SSMs achieve 4x the throughput of Transformers. For inference memory, the attention KV cache at 128K tokens is roughly 200x larger than the equivalent Mamba state (approximately 2.57 MiB per sequence).

10.4 Mixture of Experts Integration and Infinite Context

Researchers have successfully integrated Mixture of Experts (MoE) sparse routing into the Mamba-Transformer hybrid paradigm. MoE layers activate only a small subset of experts per token, significantly reducing computation while enhancing throughput. When this MoE-Mamba hybrid engine is wrapped inside the Python REPL environment of an RLM, inputs have been scaled from 2^{13} to 2^{18} tokens and beyond without Context Rot. By isolating the model from the monolithic context and relying on the selective filtering of recursive sub-instances, the hybrid architecture maintains pristine accuracy regardless of document size, establishing a true infinite-context paradigm.

Table 4: Comparative Analysis of Architectural Paradigms

Feature	Transformer	Pure Mamba	Pure RvNN/TRM	Hybrid
Complexity	$O(L^2)$ Quadratic	O(L) Linear	O(N) Tree nodes	O(L) + O(D) Seq+Depth
State Method	KV Cache	Continuous ODE	Node Aggregation	Iterative SSM
Context Limit	Fixed Window	Unbounded	Tree Depth	Infinite via REPL
Reasoning Bias	Next-Token	Filtered History	Compositional	Sequence + Logic

Parallelism	Highly Parallel	HW-Aware Scan	Topology Dep.	Hybrid Kernels

11. Implementation Guide

11.1 Selective Scan Reference

The core Mamba operation can be expressed concisely (production deployments use fused CUDA or Triton kernels):

```
def selective_scan(u, delta, A, B, C, D=None):
  dA = exp(delta.unsqueeze(-1) * A)
  dBu = delta.unsqueeze(-1) * B.unsqueeze(1) * u.unsqueeze(-1)
  h = zeros(u.shape[0], u.shape[1], A.shape[1])
  ys = []
  for t in range(u.shape[2]):
    h = dA[:,:,t] * h + dBu[:,:,t]  # state update
    y = (h * C[:,:,t].unsqueeze(1)).sum(-1)
    ys.append(y)
  return stack(ys, dim=2) + (D * u if D else 0)
```

11.2 Production Setup

For production builds, the following software stack is recommended:

```
pip install mamba-ssm>=2.2.0 causal-conv1d>=1.2.0
pip install flash-attn>=2.5.0
pip install fla-core  # Flash Linear Attention (Triton)
```

The official mamba-ssm library provides Mamba and Mamba2 modules as drop-in components. Hybrid models are natively supported via attn_layer_idx and attn_cfg parameters. For Hugging Face integration, MambaForCausalLM and Mamba2ForCausalLM are available in transformers version 4.39.0 and later. vLLM V1 now supports hybrid Mamba-Transformer models as first-class citizens with unified state management for both paged KV cache and fixed per-sequence Mamba state.

Key practical considerations include: always keeping model parameters in fp32 for training (use PyTorch AMP, not DeepSpeed fp16); using left-padding for batched generation (right-padding propagates noise through Mamba states); and noting that Mamba has scattered activation outliers that make standard per-channel quantization less effective. Rotation-assisted post-training quantization or selective

quantization (keeping attention and preceding Mamba layers in BF16 while quantizing the rest to FP8) are recommended for deployment.

12. Conclusion and Future Trajectories

The structural convergence of Mamba State Space Models and Recursive computational paradigms represents a pivotal inflection point in the architectural evolution of artificial intelligence foundation models. For nearly a decade, the field has relied almost exclusively on brute-force scaling laws applied to the single-pass, autoregressive Transformer—a strategy that has yielded impressive associative knowledge compression but has repeatedly failed to conquer complex, multi-step algorithmic reasoning and infinite-context synthesis.

The hybridization strategies analyzed in this paper—whether implemented through programmatic Recursive Language Models utilizing isolated REPL environments, latent iterative refinement in Tiny Recursive Models, or explicitly defined Tree-Structured State Space Models—demonstrate profound mathematical and structural synergy. The Mamba architecture provides an efficient, linear-time continuous engine capable of mechanistically filtering noise through its input-dependent Selection mechanism and processing massive data sequences via hardware-aware parallel SRAM scans. Recursive models provide the necessary compositional scaffold, allowing the system to break out of the 1-dimensional sequential constraint to perform hierarchical aggregation, dynamic topology mapping, and error-correcting iterative reasoning loops.

The practical path forward involves starting with the Jamba/Nemotron interleaving template (7:1 Mamba-to-attention ratio), replacing the fixed-depth stack with a Huginn-style recursive core using ALBERT-like weight sharing, adding PonderNet halting for test-time compute scaling, and training with progressive curriculum. The open-source ecosystem—mamba-ssm, FlashAttention, Flash Linear Attention, vLLM—provides all necessary implementation components.

The empirical evidence decisively supports this architectural shift. By embedding Mamba-2 operators into a recursive TRM loop, models with fewer than 10 million parameters can outperform 671-billion parameter dense autoregressive models on the most stringent logical benchmarks. Production-scale hybrid deployments from Jamba, Nemotron-H, Zamba, and Hunyuan-TurboS confirm that SSM-dominant hybrids with approximately 8% attention layers match or exceed pure Transformer quality while achieving 2–3x inference speedup and 10x or greater memory reduction.

The deeper insight emerging from this synthesis is that sequence-wise recurrence (Mamba's state propagation along tokens) and depth-wise recurrence (recursive iteration of the same block) are orthogonal dimensions of computation that serve fundamentally different purposes. Mamba compresses sequential context efficiently; recursive depth enables deliberate reasoning. Their combination creates a two-dimensional computation grid that is simultaneously more efficient and more capable than either dimension alone—and both operate with the same mathematical primitive: iterative application of a parameterized state update. This convergence suggests that the future of language model architecture lies not in choosing between efficiency and reasoning capability, but in recognizing they emerge from the same underlying computation structure applied along different axes.

References

[1] Gu, A., Goel, K., & Ré, C. (2022). Efficiently Modeling Long Sequences with Structured State Spaces. ICLR 2022.

[2] Gu, A. & Dao, T. (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv:2312.00752.

[3] Dao, T. & Gu, A. (2024). Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality. ICML 2024.

[4] Lieber, O. et al. (2024). Jamba: A Hybrid Transformer-Mamba Language Model. ICLR 2025. arXiv:2403.19887.

[5] Shaker, A. et al. (2025). Nemotron-H: A Family of Accurate and Efficient Hybrid Mamba-Transformer Models. arXiv:2504.03624.

[6] Dong, H. et al. (2024). Hymba: A Hybrid-Head Architecture for Small Language Models. NVIDIA. ICLR 2025.

[7] Glorioso, P. et al. (2024). Zamba: A Compact 7B SSM Hybrid Model. Zyphra. arXiv:2405.16712.

[8] Glorioso, P. et al. (2024). Zamba2-7B. Zyphra Technical Report.

[9] Lan, Z. et al. (2020). ALBERT: A Lite BERT for Self-supervised Learning of Language Representations. ICLR 2020.

[10] Dehghani, M. et al. (2019). Universal Transformers. ICLR 2019.

[11] Geiping, J. et al. (2025). Scaling Up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach. NeurIPS 2025.

[12] Zhu, W. et al. (2025). Ouro: Scaling Reasoning in Large Language Models via Looped Models. ByteDance.

[13] Bai, S., Kolter, J.Z., & Koltun, V. (2019). Deep Equilibrium Models. NeurIPS 2019.

[14] Banino, A. et al. (2021). PonderNet: Learning to Ponder. DeepMind. arXiv:2107.05407.

[15] Hao, S. et al. (2024). Training Large Language Models to Reason in a Continuous Latent Space. Meta. arXiv:2412.06769.

[16] Tai, K.S., Socher, R., & Manning, C.D. (2015). Improved Semantic Representations From Tree-Structured LSTM Networks. ACL 2015.

[17] De, S. et al. (2024). Griffin: Mixing Gated Linear Recurrences with Local Attention for Efficient Language Models. Google DeepMind.

[18] Peng, B. et al. (2025). Eagle and Finch: RWKV with Matrix-Valued States and Dynamic Recurrence. ICLR 2025.

[19] Sun, Y. et al. (2023). Retentive Network: A Successor to Transformer for Large Language Models. Microsoft.

[20] Ren, L. et al. (2024). Samba: Simple Hybrid State Space Models for Efficient Unlimited Context Language Modeling. Microsoft.

[21] Bick, A. et al. (2024). Transformers to SSMs: Distilling Quadratic Knowledge to Subquadratic Models (MOHAWK). NeurIPS 2024.

[22] Waleffe, R. et al. (2024). An Empirical Study of Mamba-based Language Models. NVIDIA. arXiv:2406.07887.

[23] Dao, T., Fu, D.Y., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS 2022.

[24] NVIDIA (2025). Nemotron 3: Efficient and Open Intelligence. Technical Report.

[25] IBM Research (2025). Bamba: A Hybrid Attention-State Space Model. Technical Report.

[26] Gu, A. (2025). On the Tradeoffs of SSMs and Transformers. Goomba Lab Blog.

[27] Tencent (2025). Hunyuan-TurboS: A Hybrid Mamba-Transformer Model for Large-Scale Deployment.

[28] PyTorch Foundation (2025). Hybrid Models as First-Class Citizens in vLLM. pytorch.org/blog.

[29] Zhang, A. et al. (2025). Recursive Language Models. arXiv:2512.24601.

[30] Socher, R. et al. (2013). Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank. EMNLP 2013.

[31] MambaTree: Tree Topology is All You Need in State Space Model. NeurIPS 2024.

[32] TSLM: Tree-Structured Language Modeling for Divergent Thinking. arXiv:2601.22688.

[33] Tiny Recursive Reasoning with Mamba-2 Attention Hybrid. arXiv:2602.12078.

[34] Mamba-CNN: A Hybrid Architecture for Efficient and Accurate Facial Beauty Prediction. arXiv:2509.01431.

[35] HG-Mamba: A Hybrid Geometry-Aware Bidirectional Mamba Network for Hyperspectral Image Classification. MDPI Remote Sensing, 17(13), 2234.

[36] Hybrid Mamba-Transformer MoE Architecture. Emergent Mind.

[37] Understanding the Differences in Foundation Models: Attention, State Space Models, and Recurrent Neural Networks. NeurIPS 2024.

[38] Unlocking State-Tracking in Linear RNNs Through Negative Eigenvalues. OpenReview.

[39] STree: Speculative Tree Decoding for Hybrid State Space Models. NeurIPS 2025.